

Task 1 — Submission Report

Get the Data Into a Database

Student	Ziad Suleyman
Course	MLOPS Training — 2026/2027
Trainer	Trainer Qafza
Dataset	Olist Brazilian E-Commerce (Kaggle)
Database	PostgreSQL 16.14 in Docker
Rows loaded	1,550,922 across 9 tables
Report date	21 August 2026

Every number, table and console output in this report was produced by re-running the project live on 21 Aug 2026.

0. “Done when” — self-check against the task sheet

The task defines four completion criteria. Each one is answered below with the exact section of this report that carries the evidence.

Criterion (from the task PDF)	Status	Evidence in this report
The database is running locally with the data inside	Done	\$1 (container up & healthy on localhost:5433), \$2 (row counts per table)
You can query the tables and join them	Done	\$4 — a 2-table join, a 3-table join and an aggregate, run from psql and from a Jupyter notebook
You understand the tables and the relations between them	Done	\$3 — ER diagram plus the primary/foreign keys as actually declared in the database
You understand the problem we are trying to solve	Done	\$6 — target definition in SQL, class balance, data window, and the leakage trap it hides

1. Environment — a real database, running locally

PostgreSQL 16 runs in Docker from a committed docker-compose.yml . Data lives in a named volume, so the database survives container restarts and machine reboots without re-downloading or re-ingesting anything.

```
services:
  postgres:
    image: postgres:16
    container_name: olist-postgres
    environment:
      POSTGRES_USER: olist
      POSTGRES_PASSWORD: olist123
      POSTGRES_DB: olist
    ports:
      - "5433:5432"          # 5432 is already taken by a local Postgres install
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U olist -d olist"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
```

EVIDENCE 1 — container status and server version, captured 2026-08-21 13:49 UTC

```
$ docker ps --filter name=olist-postgres
NAMES          | IMAGE          | STATUS          | PORTS
olist-postgres | postgres:16   | Up (healthy)    | 0.0.0.0:5433->5432/tcp

$ docker exec olist-postgres psql -U olist -d olist -c "select version();"
PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on x86_64-pc-linux-gnu,
compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
```

Connection string: `postgresql://olist:olist123@localhost:5433/olist`

The port is **5433**, not the default 5432, because a native PostgreSQL service already owns 5432 on this machine. Inside the container the server still listens on 5432 — only the host-side mapping differs.

Ingestion

Loading is scripted, not manual, so the whole database can be rebuilt from zero with one command:

- `schema.sql` — creates the 9 tables with column order matching the CSV headers, declares primary and foreign keys, and adds indexes on the join columns.
- `ingest.py` — applies the schema, then bulk-loads each CSV with PostgreSQL `COPY` (via `psycopg 3`), parent tables first so foreign keys never fail. Empty CSV fields are loaded as `NULL` rather than empty strings.

```
$ docker compose up -d      # start the database
$ python ingest.py          # (re)create the tables and load all 9 CSVs
```

2. The data is inside — row counts per table

Run against the live database at capture time. This is the direct answer to “the database is running locally with the data inside”.

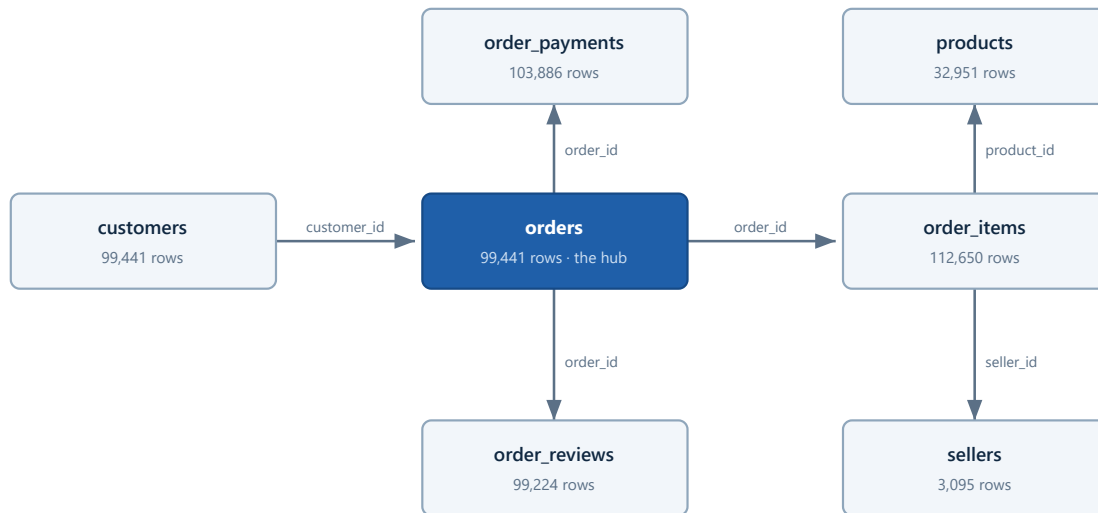
EVIDENCE 2 — test_queries.sql , query 1: row count per table

table_name	count
customers	99441
geolocation	1000163
order_items	112650
order_payments	103886
order_reviews	99224
orders	99441
product_category_name_translation	71
products	32951
sellers	3095
(9 rows)	

Table	Rows	What one row is
orders	99,441	One order — carries the four timestamps the whole problem depends on
customers	99,441	One order’s customer record — <i>not</i> one person; the person-level id is customer_unique_id
order_items	112,650	One product line inside an order
order_payments	103,886	One payment — a single order can be split across several
order_reviews	99,224	One review, with score and free-text comment
products	32,951	One product: category, weight, dimensions, photo count
sellers	3,095	One seller and their location
geolocation	1,000,163	One lat/lng point for a zip-code prefix (contains duplicates)
product_category_name_translation	71	Portuguese → English category name
Total	1,550,922	9 tables

3. Tables and relations

The schema is a star around `orders` . `order_items` is the bridge that connects an order to both the products and the sellers it involves.



Two tables stand outside the graph: `geolocation` (1,000,163 rows, joined by zip-code prefix, no key) and `product_category_name_translation` (71 rows, joined by category name).

Keys as actually declared in the database

EVIDENCE 3 — primary and foreign keys read back from `information_schema`

table_name	type	col	references
customers	PRIMARY KEY	customer_id	
orders	PRIMARY KEY	order_id	
orders	FOREIGN KEY	customer_id	customers
order_items	PRIMARY KEY	order_id, order_item_id	
order_items	FOREIGN KEY	order_id	orders
order_items	FOREIGN KEY	product_id	products
order_items	FOREIGN KEY	seller_id	sellers
order_payments	PRIMARY KEY	order_id, payment_sequential	
order_payments	FOREIGN KEY	order_id	orders
order_reviews	PRIMARY KEY	review_id, order_id	
order_reviews	FOREIGN KEY	order_id	orders
products	PRIMARY KEY	product_id	
sellers	PRIMARY KEY	seller_id	
product_category_name_translation	PRIMARY KEY	product_category_name	

(17 constraint rows)

Note the two composite primary keys. `order_items` is keyed on `(order_id, order_item_id)` because one order holds several lines, and `order_reviews` on `(review_id, order_id)` because a single review can cover more than one order — `review_id` alone is **not** unique, which is a common source of duplicated rows when people join it carelessly.

4. Querying and joining

All three queries below come from `test_queries.sql` and were re-run at capture time. The same queries also run from the Jupyter notebook through SQLAlchemy (\$5).

4.1 Two-table join — orders with their customers

```
SELECT o.order_id, o.order_status, o.order_purchase_timestamp,  
       c.customer_city, c.customer_state  
FROM orders o  
JOIN customers c USING (customer_id)  
LIMIT 10;
```

EVIDENCE 4 — output

order_id	order_status	order_purchase_timestamp	customer_city	customer_state
5f79b5b0931d63f1a42989eb65b9da6e	delivered	2017-11-14 16:08:26	osasco	SP
a44895d095d7e0702b6a162fa2dbeced	delivered	2017-07-16 09:40:32	itapecerica	MG
316a104623542e4d75189bb372bc5f8d	delivered	2017-02-28 11:06:43	nova venecia	ES
5825ce2e88d5346438686b0bba99e5ee	delivered	2017-08-16 13:09:20	mendonca	MG
0ab7fb08086d4af9141453c91878ed7a	delivered	2018-04-02 13:42:17	sao paulo	SP
cd3558a10d854487b4f907e9b326a4fc	delivered	2017-04-12 08:35:12	valinhos	SP
07f6c3baf9ac86865b60f640c4f923c6	delivered	2018-03-02 17:47:40	niteroi	RJ
8c3d752c5c02227878fae49aeaddbdf7	delivered	2017-12-18 11:08:30	rio de janeiro	RJ
fa906f338cee30a984d0945b3832e431	delivered	2017-09-17 16:04:44	ijui	RS
9b961b894e797f63622137ff7eb1c1af	delivered	2018-08-11 12:14:35	oliveira	MG

(10 rows)

4.2 Three-table join — order items with product category and seller state

```
SELECT oi.order_id, p.product_category_name, s.seller_state, oi.price  
FROM order_items oi  
JOIN products p USING (product_id)  
JOIN sellers s USING (seller_id)  
LIMIT 10;
```

EVIDENCE 5 — output

order_id	product_category_name	seller_state	price
00010242fe8c5a6d1ba2dd792cb16214	cool_stuff	SP	58.90
00018f77f2f0320c557190d7a144bdd3	pet_shop	SP	239.90
000229ec398224ef6ca0657da4fc703e	moveis_decoracao	MG	199.00
00024acbcdcf0a6daa1e931b038114c75	perfumaria	SP	12.99
00042b26cf59d7ce69dfabb4e55b4fd9	ferramentas_jardim	PR	199.90
00048cc3ae777c65dbb7d2a0634bc1ea	utilidades_domesticas	SP	21.90
00054e8431b9d7675808bcb819fb4a32	telefonica	SP	19.90
000576fe39319847cbb9d288c5617fa6	ferramentas_jardim	SP	810.00
0005a1a1728c9d785b8e2b08b904576c	beleza_saude	SP	145.95
0005f50442cb953dcd1d21e1fb923495	livros_tecnicos	SP	53.99

(10 rows)

4.3 The query that defines the problem

```

SELECT count(*) AS delivered_orders,
       count(*) FILTER (WHERE order_delivered_customer_date
                             > order_estimated_delivery_date) AS late_orders,
       round(100.0 * count(*) FILTER (WHERE order_delivered_customer_date
                                         > order_estimated_delivery_date)
            / count(*), 2) AS late_pct
FROM orders
WHERE order_status = 'delivered'
      AND order_delivered_customer_date IS NOT NULL;

```

EVIDENCE 6 — output

delivered_orders	late_orders	late_pct
96470	7826	8.11

(1 row)

5. The same thing from a notebook

Step 3 of the task asks for a notebook or terminal connection. Both are covered: `psql` inside the container for the outputs above, and `olist_explore.ipynb` for a pandas-side connection. The notebook was executed top-to-bottom on 2026-08-21 and is submitted **with its outputs saved**, so it can be read without a running database.

```
from sqlalchemy import create_engine, text
import pandas as pd

engine = create_engine("postgresql+psycopg://olist:olist123@localhost:5433/olist")
pd.read_sql(text("SELECT * FROM orders LIMIT 5"), engine)
```

The eight executed cells cover: server version · listing the tables from `information_schema` · row count per table · a sample of `orders` · the two-table join · the three-table join · the late-delivery aggregate.

The connection URL uses the `postgresql+psycopg://` prefix — SQLAlchemy needs the driver named explicitly to use psycopg 3; the plain `postgresql://` form falls back to psycopg2, which is not installed.

6. The problem we are solving

Question: at the moment an order is placed, will it be delivered *late* or *on time*?

Target, in SQL:

```
is_late = (order_delivered_customer_date > order_estimated_delivery_date)
```

What the target looks like

Property	Value	Why it matters
Delivered orders with a delivery date	96,470	The usable population — 97.0% of all orders have status <code>delivered</code>
Late orders	7,826	The positive class
Late rate	8.11%	Imbalanced — accuracy is useless here; ROC-AUC / PR-AUC / recall at a fixed alert budget are the honest metrics
Data window	2016-09-04 → 2018-10-17	Just over two years — enough for a chronological train/test split
Avg actual delivery	12.6 days	The store promises about 11 days more than it typically needs, and still misses in 8% of cases
Avg promised delivery	23.7 days	

Why the problem is worth solving

EVIDENCE 7 — review score of late vs on-time orders

is_late	orders	avg_review_score	pct_1_2_star
false	88653	4.29	9.2
true	7700	2.57	54.0

A late delivery costs the store **1.7 stars** on average, and the share of 1–2 star reviews jumps from 9.2% to **54.0%**. Being late is not a metric — it is over half of those customers publicly punishing the store. That is what makes an

early-warning model worth building.

The trap hidden in this table

`orders` holds four delivery-related timestamps, and they do **not** all exist at prediction time:

Column	Known at purchase?	Verdict
<code>order_purchase_timestamp</code>	Yes	Safe feature
<code>order_estimated_delivery_date</code>	Yes	Safe — and it defines the target
<code>order_approved_at</code>	No	Leakage at purchase time
<code>order_delivered_carrier_date</code>	No	Leakage at purchase time — but available later
<code>order_delivered_customer_date</code>	No	This <i>is</i> the answer — never a feature

النسخة العربية

تقرير تسليم التاسك الأول — نفس المحتوى وبالترتيب نفسه

٠. مطابقة معايير الإنجاز المذكورة في التاسك

حدّد ملف التاسك أربعة معايير للإنجاز، وفيما يلي كل معيار مع موضع دليله في هذا التقرير.

المعيار كما ورد في التاسك	الحالة	الدليل
قاعدة البيانات شغّالة محليًا والبيانات بداخلها	مُنجز	القسم ١ (الحاوية تعمل بحالة healthy على localhost:5433) والقسم ٢ (عدد الصفوف في كل جدول)
تستطيع الاستعلام عن الجداول ودمجها بـ join	مُنجز	القسم ٤ — دمج جدولين ودمج ثلاثة جداول واستعلام تجميعي، مُنفّذة من psql ومن دفتر Jupyter
تفهم الجداول والعلاقات بينها	مُنجز	القسم ٣ — مخطط العلاقات مع المفاتيح الأساسية والأجنبية كما هي مُعرّفة فعليًا داخل القاعدة
تفهم المشكلة التي نحاول حلّها	مُنجز	القسم ٦ — تعريف الهدف بـ SQL، وتوازن الفئات، ونافذة البيانات، والفجّ المخفي فيها

١. البيئة — قاعدة بيانات حقيقية تعمل محليًا

تعمل PostgreSQL 16 داخل Docker انطلاقًا من ملف `docker-compose.yml` مرفوع مع المشروع، والبيانات مخزّنة في volume باسم `pgdata`، أي أن القاعدة تنجو من إعادة تشغيل الحاوية أو الجهاز دون الحاجة لإعادة التحميل أو إعادة الإدخال.

الدليل ١ — حالة الحاوية وإصدار الخادم، مأخوذة في 21-08-2026 الساعة 13:49 UTC

```
$ docker ps --filter name=olist-postgres
NAMES          | IMAGE          | STATUS          | PORTS
olist-postgres | postgres:16   | Up (healthy)    | 0.0.0.0:5433->5432/tcp

$ docker exec olist-postgres psql -U olist -d olist -c "select version();"
PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on x86_64-pc-linux-gnu
```

سلسلة الاتصال: `postgresql://olist:olist123@localhost:5433/olist`

المنفذ 5433 وليس 5432 لأن خدمة PostgreSQL محلية تحتلّ المنفذ الافتراضي على هذا الجهاز. داخل الحاوية يبقى الخادم يستمع على 5432، والاختلاف في التوجيه من جهة المضيف فقط.

الإدخال (Ingestion)

- `schema.sql` — ينشئ الجداول التسعة بترتيب أعمدة مطابق لترويسات ملفات CSV، ويعرّف المفاتيح الأساسية والأجنبية، ويضيف فهرس على أعمدة الربط.
 - `ingest.py` — يطبّق المخطط ثم يحمّل كل ملف CSV دفعةً واحدة عبر أمر `COPY` (بواسطة 3 `psycopg`)، مع تحميل الجداول الأب أولاً حتى لا تفشل المفاتيح الأجنبية. الحقول الفارغة تُحمّل كـ `NULL` لا كسلاسل فارغة.
- أمران فقط يعيدان بناء القاعدة من الصفر: `docker compose up -d` ثم `python ingest.py`.

٢. البيانات موجودة فعليًا — عدد الصفوف في كل جدول

الجدول	الصفوف	ما الذي يمثّله الصف الواحد
orders	99,441	طلب واحد — يحمل الطوابيع الزمنية الأربعة التي تقوم عليها المشكلة كلها
customers	99,441	سجلّ زبون مرتبط بطلب — وليس شخصًا واحدًا؛ المعرّف على مستوى الشخص هو customer_unique_id
order_items	112,650	سطر منتج واحد داخل الطلب
order_payments	103,886	دفعة واحدة — الطلب الواحد قد يُقسّم على عدة دفعات
order_reviews	99,224	تقييم واحد بدرجته ونصّه
products	32,951	منتج واحد: الفئة والوزن والأبعاد وعدد الصور
sellers	3,095	بائع واحد وموقعه
geolocation	1,000,163	إحداثية لبادئة رمز بريدي (يحتوي تكرارات)
product_category_name_translation	71	ترجمة اسم الفئة من البرتغالية إلى الإنكليزية
المجموع	1,550,922	تسعة جداول

٣. الجداول والعلاقات

المخطط نجميّ يدور حول جدول orders ، و جدول order_items هو الجسر الذي يربط الطلب بالمنتجات والبائعين معًا. المخطط البياني معروض في القسم ٣ من النسخة الإنكليزية.

أهم العلاقات: orders.customer_id → customers ، و order_items يربط كل سطر بـ product_id و seller_id ، والدفعات والتقييمات تتفرّع من orders.order_id . ويقف خارج المخطط جدولان: geolocation (يُربط ببادئة الرمز البريدي وبلا مفتاح) و product_category_name_translation (يُربط باسم الفئة).

مفتاحان مركّبان يستحقان الانتباه: مفتاح order_items هو (order_id, order_item_id) لأن الطلب يحوي أكثر من سطر، ومفتاح order_reviews هو (review_id, order_id) لأن التقييم الواحد قد يغطي أكثر من طلب — أي أن review_id وحده ليس فريدًا، وهذا مصدر شائع لتكرار الصفوف عند الدمج غير المنتبه.

٤. الاستعلام والدمج

الاستعلامات الثلاثة مأخوذة من test_queries.sql وأُعيد تنفيذها لحظة إعداد هذا التقرير، ومخرجاتها الكاملة معروضة في القسم ٤ من النسخة الإنكليزية:

1. دمج جدولين — الطلبات مع الزبائن عبر customer_id (١٠ صفوف).
2. دمج ثلاثة جداول — عناصر الطلب مع المنتج والبائع عبر product_id و seller_id (١٠ صفوف).
3. استعلام المشكلة — عدد الطلبات المتأخرة ونسبتها.

الدليل ٦ — نتيجة استعلام المشكلة

```
delivered_orders | late_orders | late_pct
-----+-----+-----
          96470 |         7826 |      8.11
```

٥. الشيء نفسه من دفتر Jupyter

الخطوة الثالثة من التاسك تطلب الاتصال من دفتر أو من الطرفية، والاثنتان مغطيان: `psql` داخل الحاوية للمخرجات أعلاه، ودفتر `olist_explore.ipynb` للاتصال عبر `pandas`. نُقِّذ الدفتر بالكامل من أوله إلى آخره بتاريخ 21-08-2026 ويُسلّم ومخرجاته محفوظة داخله، أي يمكن قراءته دون تشغيل القاعدة.

تستخدم سلسلة الاتصال البادئة `postgres://postgres:psycopg@localhost:5432/olist` لأن `SQLAlchemy` يحتاج تسمية المُشغِّل صراحةً لاستخدام `psycopg3`؛ الصيغة العادية `postgres://` ترجع افتراضياً إلى `psycopg2` وهو غير مثبّت.

6. المشكلة التي نحلّها

السؤال: لحظة إتمام الطلب، هل سيصل متأخراً أم في الوقت؟

is_late = (order_delivered_customer_date > order_estimated_delivery_date)

الخاصية	القيمة	لماذا تهّم
الطلبات المسلمة ولها تاريخ تسليم	96,470	المجتمع القابل للاستخدام — ٩٧٪ من الطلبات حالتها delivered
الطلبات المتأخرة	7,826	الفئة الموجبة
نسبة التأخير	8.11%	عدم توازن — الدقة (accuracy) عديمة الفائدة هنا؛ المقاييس الصادقة هي ROC-AUC و PR-AUC والاستدعاء عند ميزانية إنذار محددة
نافذة البيانات	2016-09-04 ← 2018-10-17	سنتان وأكثر — تكفي لتقسيم زمني بين تدريب واختبار
متوسط التسليم الفعلي	12.6 يومًا	المتجر يعد بـ ١١ يومًا زيادة عمّا يحتاجه عادةً، ومع ذلك يتأخر في ٨٪ من الحالات
متوسط الموعد الموعود	23.7 يومًا	

لماذا تستحق المشكلة الحل

الطلب المتأخر يُسقط متوسط التقييم من 4.29 إلى 2.57 نجمة، وترتفع نسبة التقييمات السيئة (نجمة أو نجمتان) من 9.2٪ إلى 54.0٪. أي أن أكثر من نصف الزبائن المتأخرة طلباتهم يعاقبون المتجر علنًا — وهذا ما يجعل نموذج الإنذار المبكر مجديًا تجاريًا لا أكاديميًا فقط.

الفخّ المخفي في جدول الطلبات

جدول `orders` يحوي أربعة طوابع زمنية متعلقة بالتسليم، وهي ليست كلها متاحة لحظة التنبؤ:

العمود	متاح لحظة الشراء؟	الحكم
order_purchase_timestamp	نعم	ميزة آمنة
order_estimated_delivery_date	نعم	آمنة — وهي التي تعرّف الهدف
order_approved_at	لا	تسريب لحظة الشراء
order_delivered_carrier_date	لا	تسريب لحظة الشراء — لكنه متاح لاحقًا
order_delivered_customer_date	لا	هذا هو الجواب نفسه — لا يكون ميزة أبدًا